

Distributed Systems

Lecture20: Gossip-Based Coordination

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Gossip-based Coordination: Aggregation

- **Typical apps:**
  - **Data dissemination:** Perhaps the most important one. Note that there are many variants of dissemination.
  - **Aggregation:** Let every node P_i maintain a variable v_i . When two nodes gossip, they each reset their variable to

$$v_i, v_j \leftarrow (v_i + v_j)/2$$

  - Result: in the end each node will have computed the average

$$\bar{v} = \Sigma_i v_i / N.$$

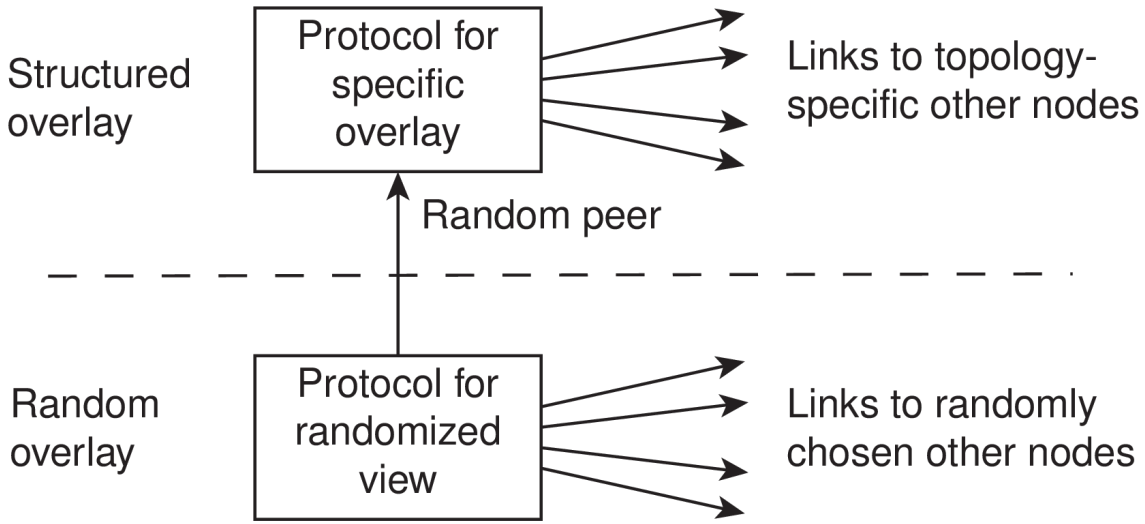
  - What happens in the case that initially $v_i = 1$ and $v_j = 0, j \neq i$?

Gossip-based Coordination: Peer Sampling

- **Problem**
- For many gossip-based applications, you need to select a peer uniformly at random from the entire network. In principle, this means you need to know all other peers. Impossible?
- **Basics**
  - Each node maintains a list of c references to other nodes
  - Regularly, pick another node at random (from the list), and exchange roughly c/2 references
  - When the application needs to select a node at random, it also picks a random one from its local list.
- **Statistically**, it turns out that the selection of a peer from the local list is indistinguishable from selecting uniformly at random peer from the entire network

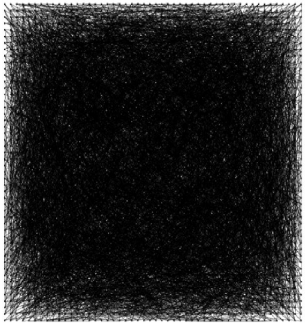
Gossip-based Overlay Construction

- Maintain two local lists of neighbors. The lowest is used for providing a peer-sampling service; the highest list is used to carefully select application-dependent neighbors.

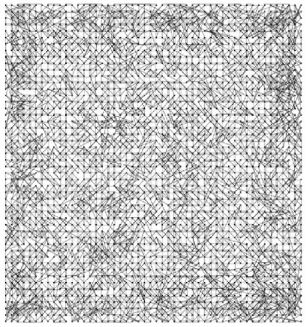


Gossip-based Overlay Construction: a 2D Torus

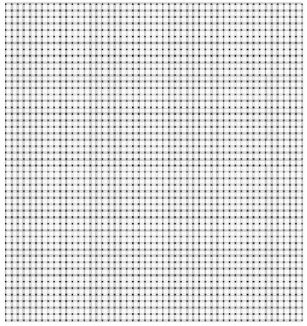
- Consider a logical N ×N grid, with a node on each point of the grid.
  - Every node must maintain a list of c nearest neighbors
  - Distance between node at (a_1, a_2) and (b_1, b_2) is $d_1 + d_2$, with
    $$d_i = \min(N - |a_i - b_i|, |a_i - b_i|)$$
  - Every node picks a random other node from its lowest-level list, and keeps only the closest one in its top-level list.
  - Once every node has picked and selected a random node, we move to the next round



Start (N = 50)



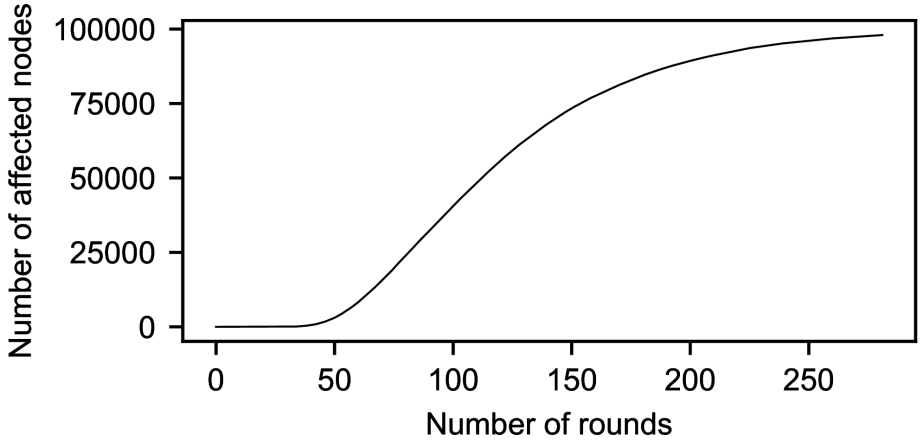
After 5 Rounds



After 20 Rounds

Secure Gossiping

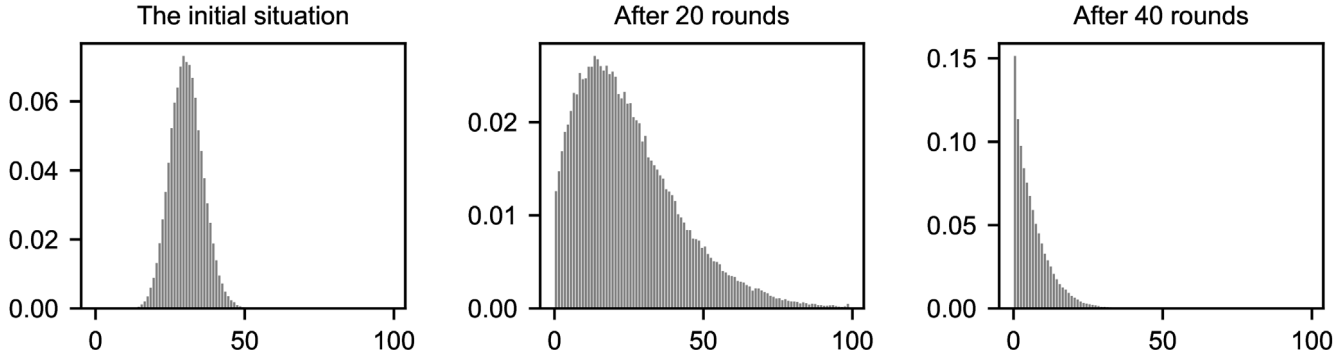
- **Dramatic attack**
- Consider when exchanging references, a set of colluding nodes systematically returns links only to each other ⇒ we are dealing with hub attack.



- **Situation**
- A network with 100,000 nodes, a local list size c = 30, and only 30 attackers. The y-axis shows the number of nodes with links only to the attackers. After less than 300 rounds, the attackers have full control.

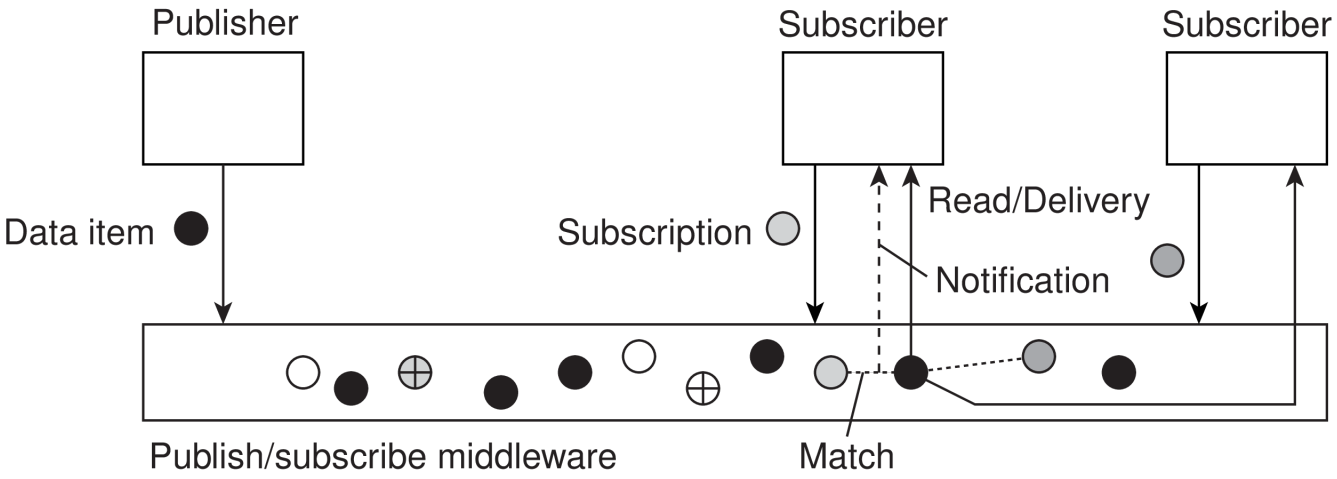
A Solution: Gathering Statistics

- This is what measuring indegree distributions tells us: which fraction of nodes (y-axis) have how many other nodes pointing to them (x-axis)?



- **Basic approach**
- When a benign node initiates an exchange, it may either use the result for gathering statistics, or for updating its local list. An attacker is in limbo: will its response be used for statistical purposes or for functional purposes?
- When gathering statistics may reveal colluders, **a colluding node will be forced to behave according to the protocol.**

Distributed Event Matching

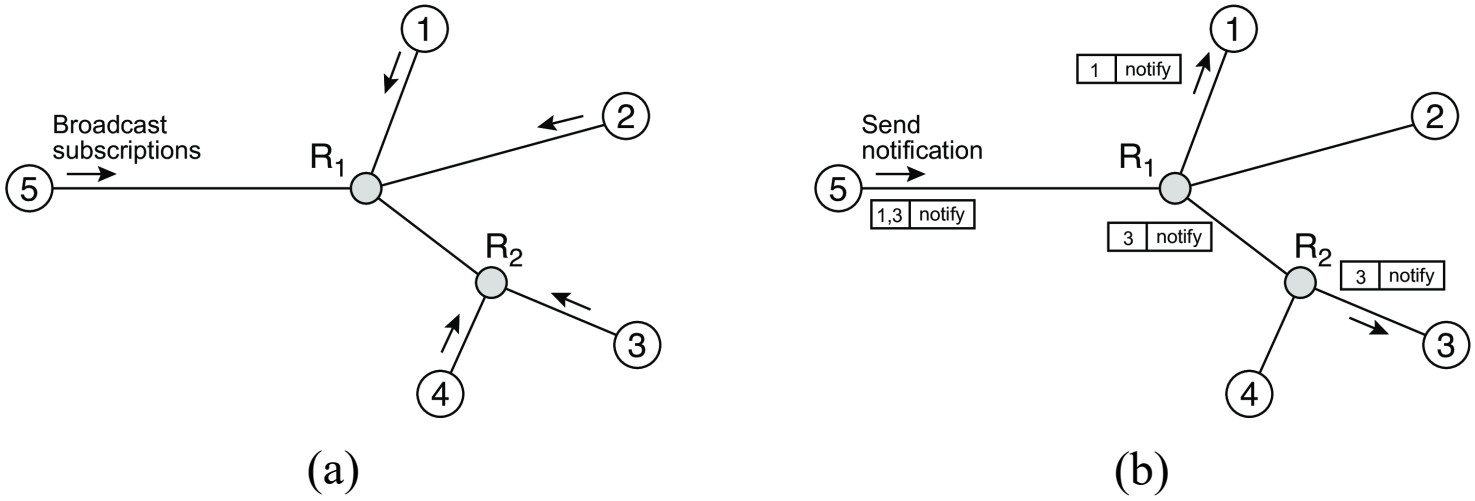


- **Principle**
  - A process specifies in which events it is interested (subscription S)
  - When a process publishes a notification N we need to see whether S matches N.
- **Hard part**
  - Implementing the match function in a scalable manner.

General Approach

- **What is needed:**
  - sub2node(S): map a subscription S to a nonempty subset S of servers
  - not2node(N): map a notification N to a nonempty subset N of servers Make sure that $S \cap N \neq \emptyset$
- **Observations**
  - Centralized solution is simple: $S = N = \{s\}$, i.e. a single server.
  - Topic-based publish-subscribe is also simple: each S and N is tagged with a single topic; each topic is handled by a single server (a rendezvous node). Several topics may be handled by same server).
  - Content-based publish-subscribe is tough: a subscription takes the form (attribute, value) pair, with example values:
    - range: "$1 \leq x < 10$"
    - containment: "$x \in \{red, blue\}$"
    - prefix and suffix expressions: "url.startswith("https")"

Selective Routing



a) First broadcast subscriptions
b) Forward notifications only to relevant rendezvous nodes

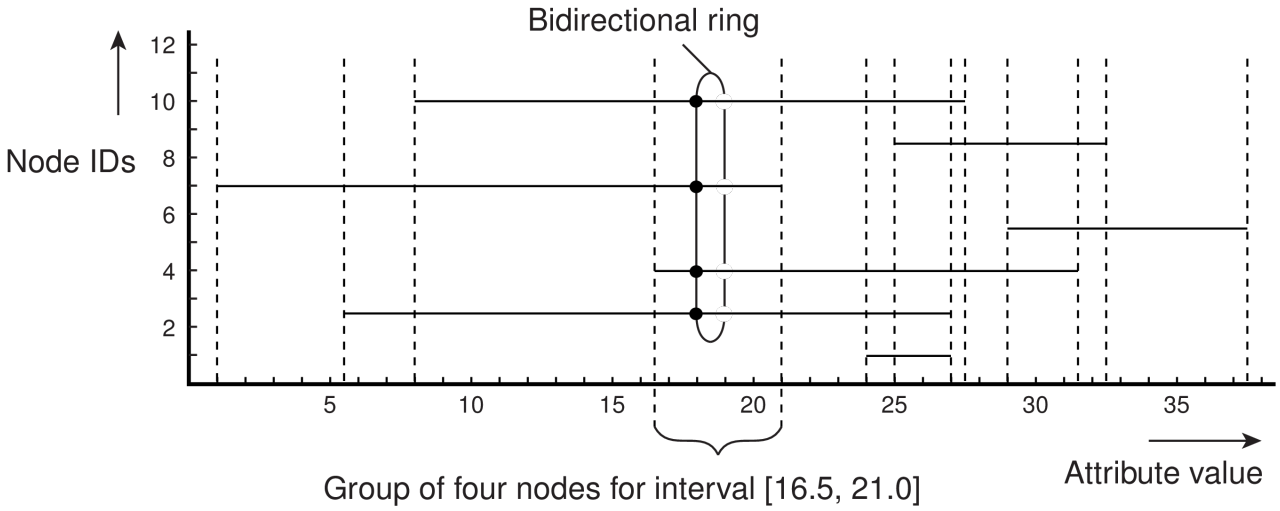
- **Example of a (partially filled) routing table**

Interface	Filter
To node 3	a $\in [0,3]$
To node 4	a $\in [2,5]$
Toward router R_1	(unspecified)

Gossiping: Sub-2-Sub

- **Basics**
  - Goal: To realize scalability, make sure that subscribers with the same interests form just a single group
  - Model: There are *N* attributes $a_1, \ldots, a_N$. An attribute value is always (mappable to) a floating-point number.
  - Subscription: Takes forms such as $S = \langle a_1 \rightarrow 3.0, a_4 \rightarrow [0.0, 0.5)\rangle$: a_1 should be 3.0; a_4 should lie between 0.0 and 0.5; other attribute values don't matter.
- **Observations**
  - A subscription S_i specifies a subset $\boldsymbol{S_i}$ in a *N*-dimensional space.
  - We are interested only in notifications that fall into $\bar{S} = \cup S_i$.
- **Goal**
  - Partition $\bar{S}$ into M disjoint subspaces $\bar{S}_1, \ldots, \bar{S}_M$ such that
  - Partitioning: $\forall k \neq m : \bar{S}_k \cap \bar{S}_m \neq 0$ and $\cup_m \bar{S}_m = \bar{S}$
  - Subscription coverage: $(\bar{S}_m \cap S_i \neq 0) \Rightarrow (S_m \subseteq S_i)$

Gossiping: Sub-2-Sub



- Consider a single attribute
  - Nodes regularly exchange their subscriptions through gossiping
  - An intersection between two nodes leads to a mutual reference
  - If $S_{ijk} = S_i \cap S_j \cap S_k \neq \emptyset$ and $S_{ij} - S_{ijk} \neq \emptyset$, then:
    - nodes i, j, k are grouped into a single overlay network (for S_{ijk})
    - nodes i, j are grouped into a single overlay network (for $S_{ij} - S_{ijk}$)

Secure Publish-subscribe

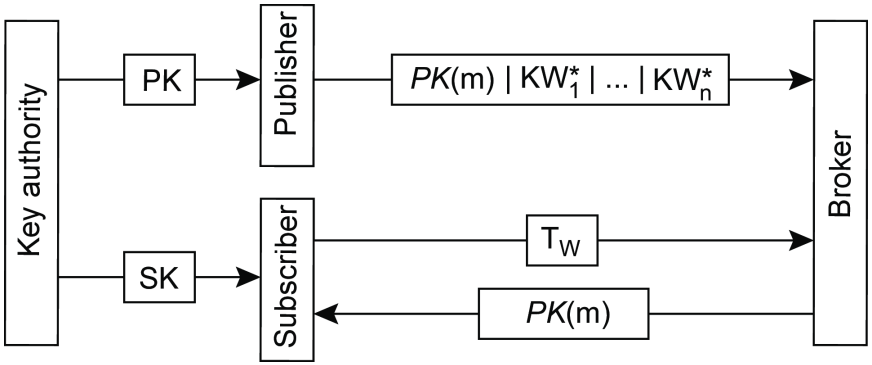
- We are facing nasty dilemma's
  - Referential decoupling: messages should be able to flow from a publisher to subscribers while guaranteeing mutual anonymity ⇒ we cannot set up a secure channel.
  - Not knowing where messages come from imposes integrity problems.
  - Assuming a trusted broker may easily be practically impossible, certainly when dealing with sensitive information ⇒ we now have a routing problem.
- Solution
  - Allow for searching (and matching) on encrypted data, without the need for decryption.
  - PEKS: accompany encrypted messages with a collection of (again encrypted) keywords and search for matches on keywords.

Public-Key Encryption with Keyword Search (PEKS)

- Basics
  - Use a public key PK , message m and its n keywords $KW_1, \ldots, KW_n$ are stored at a server as the message m$*$:

$$\mathrm{m}^* = [PK(m)|PEKS(PK, KW_1)|PEKS(PK, KW_2)| \cdots |PEKS(PK, KW_n)]$$

  - A subscriber gets the accompanying secret key.
  - For each keyword KW_i , a trapdoor T_{KW_i} is generated: $T_w(m^*)$ will return $true\ iff\ W \in \{KW_1, \ldots, KW_n\}$.



$$KW_i^* = PEKS(PK, KW_i)$$

Chapter Wrap-Up

In the next chapter, we will discuss how the entities are identified in a distributed systems. We also take a look at below topics:

- Naming Basics,
- Flat Naming,
- Structured Naming,
- Named Data Networking.